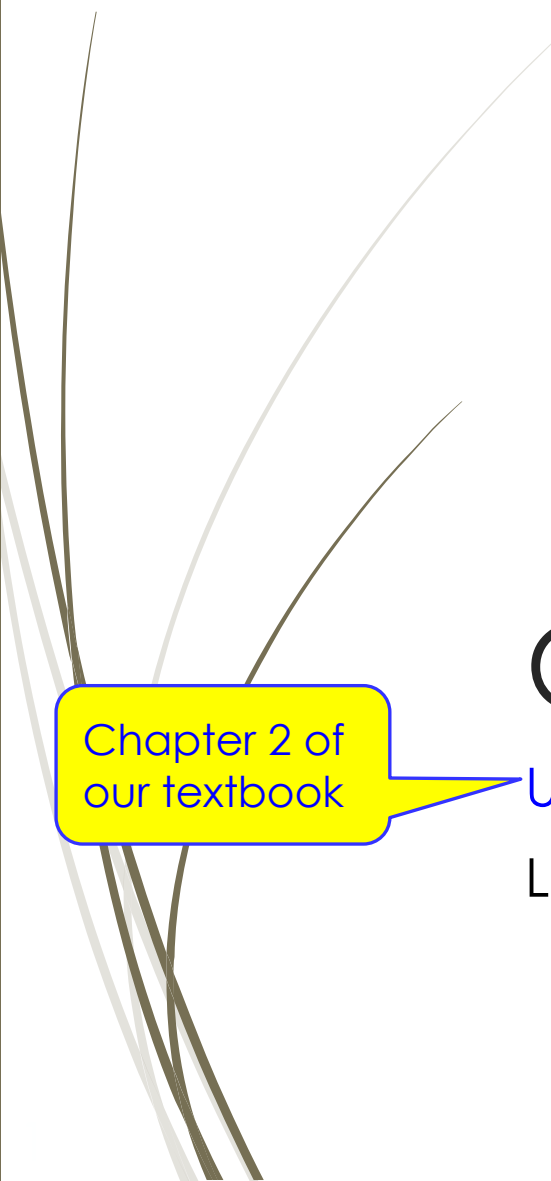




Chapter 2 of
our textbook

CMPT 295

Unit - Data Representation

Lecture 2 – Storing data in memory

Last Lecture

- ✓ What is CMPT 295?
 - ✓ What shall we learn in CMPT 295?
 - ✓ What should we already know?
 - ✓ Which resources do we have to help us learn all this?
- ✓ Activity
 - ✓ Thank you for participating in the Lecture 1 Activity!
 - ✓ I have now posted a video from Robert Seacord - CERT® Secure Coding Initiative in which he reveals the answer to the Code question of our Lecture 1 Activity! Check it out!
- ✓ My approach to teaching
- ✓ Questions

Most of this
is review

Today's Menu

➤ Storing data in memory

➤ “Under the Hood” - **von Neumann** architecture

➤ Bits and bytes in memory

Note that we do not know yet
what data these bits represent!

➤ How to diagram memory (used in this course and other references)

reviewed in Lab 0 ➤ How to express memory content -> **using binary and hexadecimal numeral system**

➤ Order of bytes in memory -> **endian**

➤ Bit manipulation – bitwise operations

➤ Boolean algebra + shifting

We shall start giving
meaning to bits -> what
data they represent.

➤ What kind of data do bits (in memory) represent -> **encoding scheme**

➤ Bits representing (encoding) integral numbers

reviewed in Lab 1 ➤ Unsigned and signed

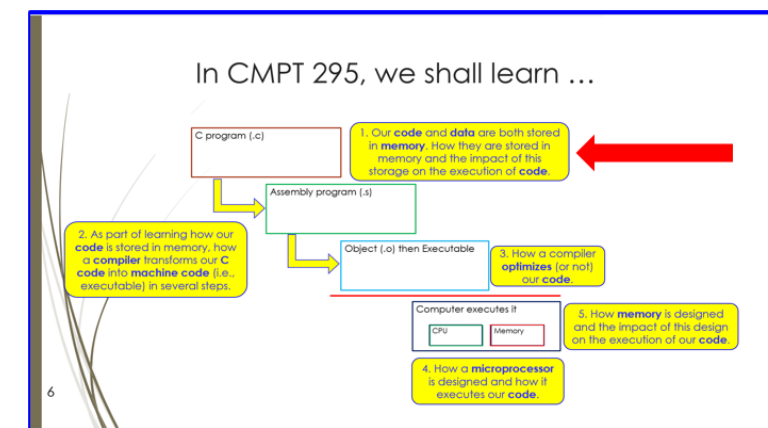
➤ Converting, expanding and truncating – Impact of fixed size memory

➤ Arithmetic operations

➤ Bits representing (encoding) real numbers

➤ IEEE floating point representation

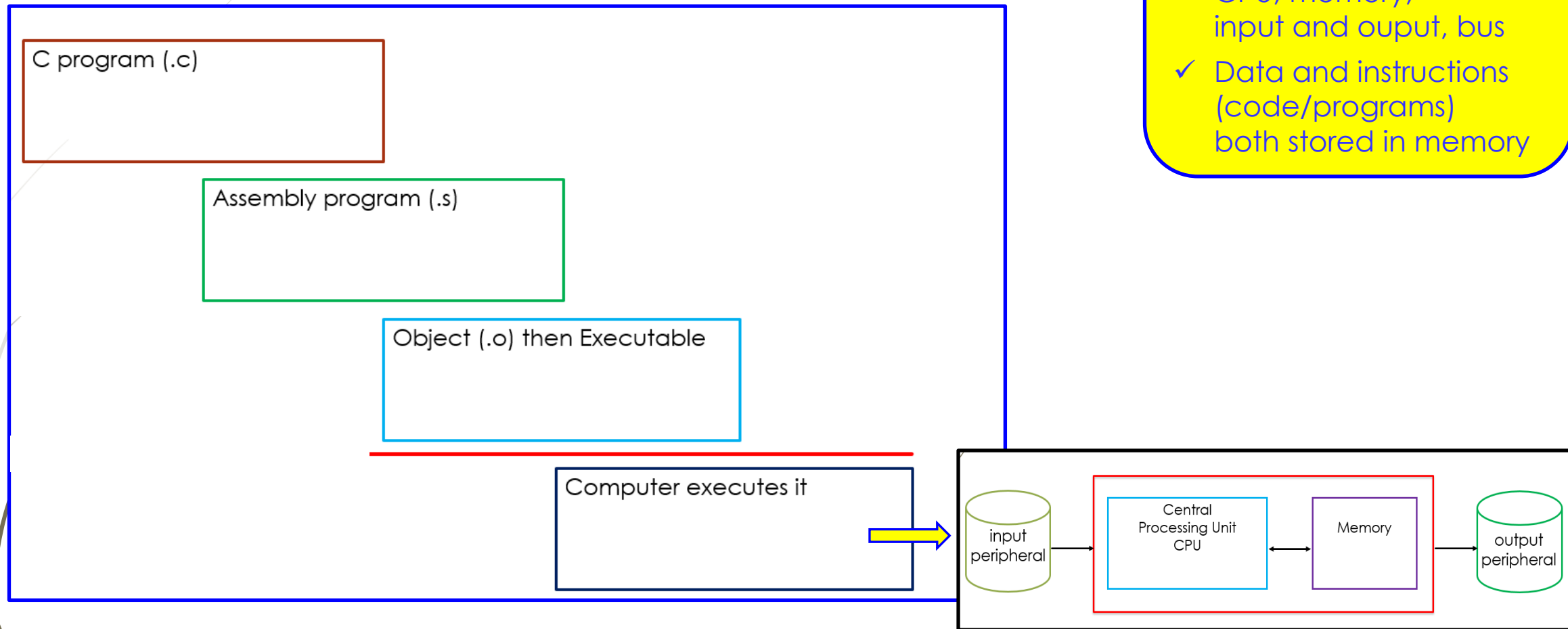
➤ Floating point in C – casting, rounding, addition, ...



“Under the hood” - **von Neumann** architecture

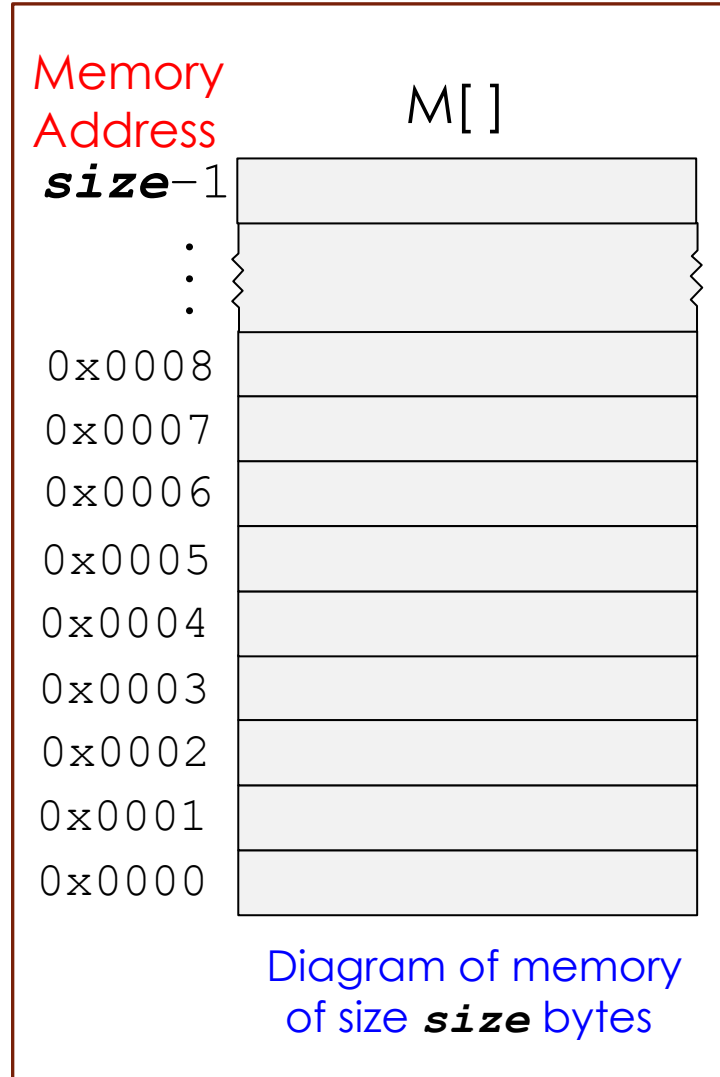
Architecture of most computers featuring:

- ✓ CPU, memory, input and output, bus
- ✓ Data and instructions (code/programs) both stored in memory



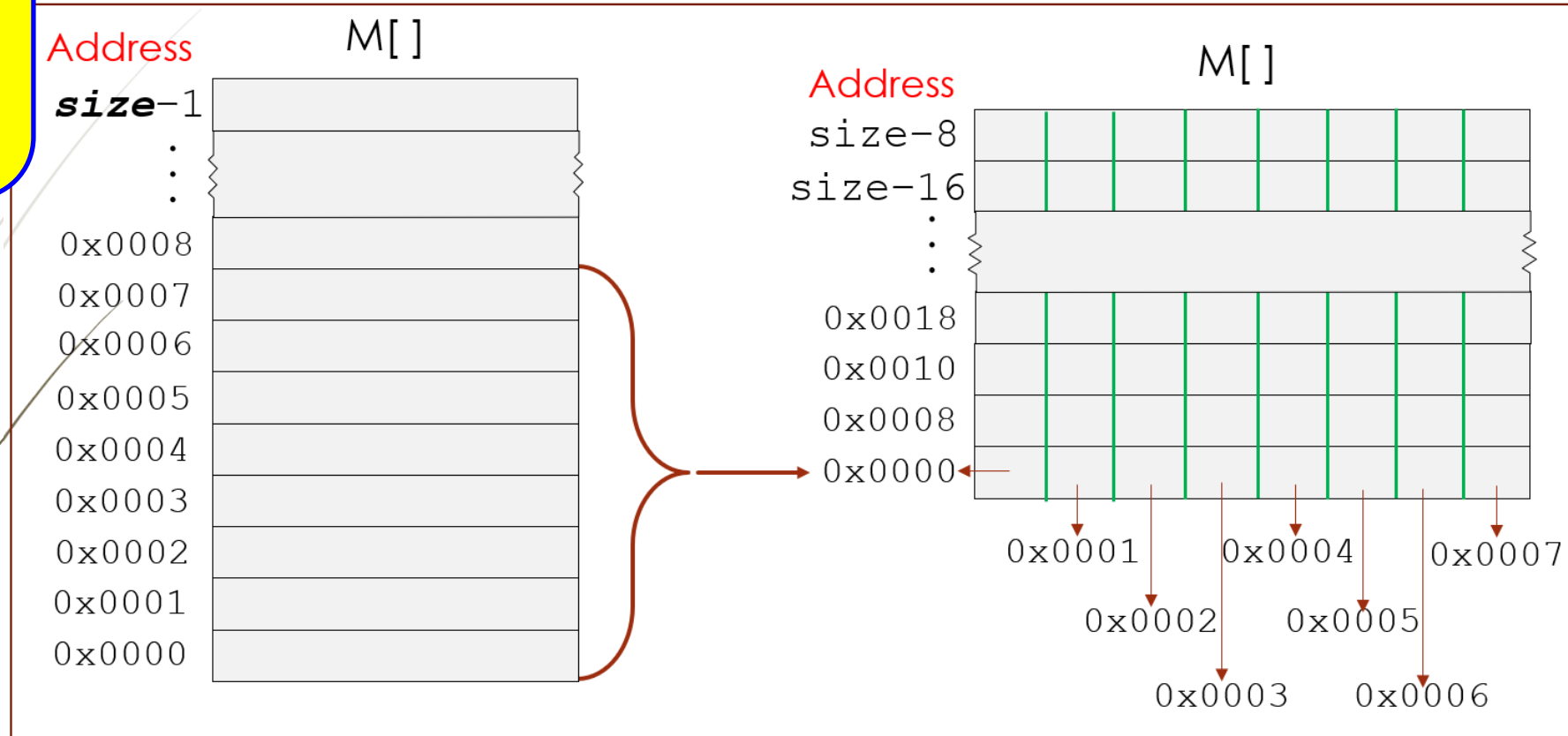
How to diagram **memory**

- Seen as a linear (contiguous) array of bytes
- 1 byte (8 bits) is smallest addressable unit (*chunk*) of memory
 - Each byte has a **unique** memory address
 - Memory that allows each byte to be accessed is called **byte-addressable** memory
- Microprocessor reads a **word** at a time from memory
 - **word size**: # of bytes (or bits) being read @ a time
- **Questions:**
 1. If word size is 8 bits, how many bytes are read at a time from memory?
Answer: _____
 2. If word size is 64 bits, how many bytes are read at a time from memory?
Answer: _____
 3. If a computer can read 4 bytes at a time, its word size is _____ bits .



NOTE: **Compressed** refers to the way we diagram memory. It has no impact on the content of memory, i.e., we are not compressing the content of memory. Nothing to do with compression algorithm.

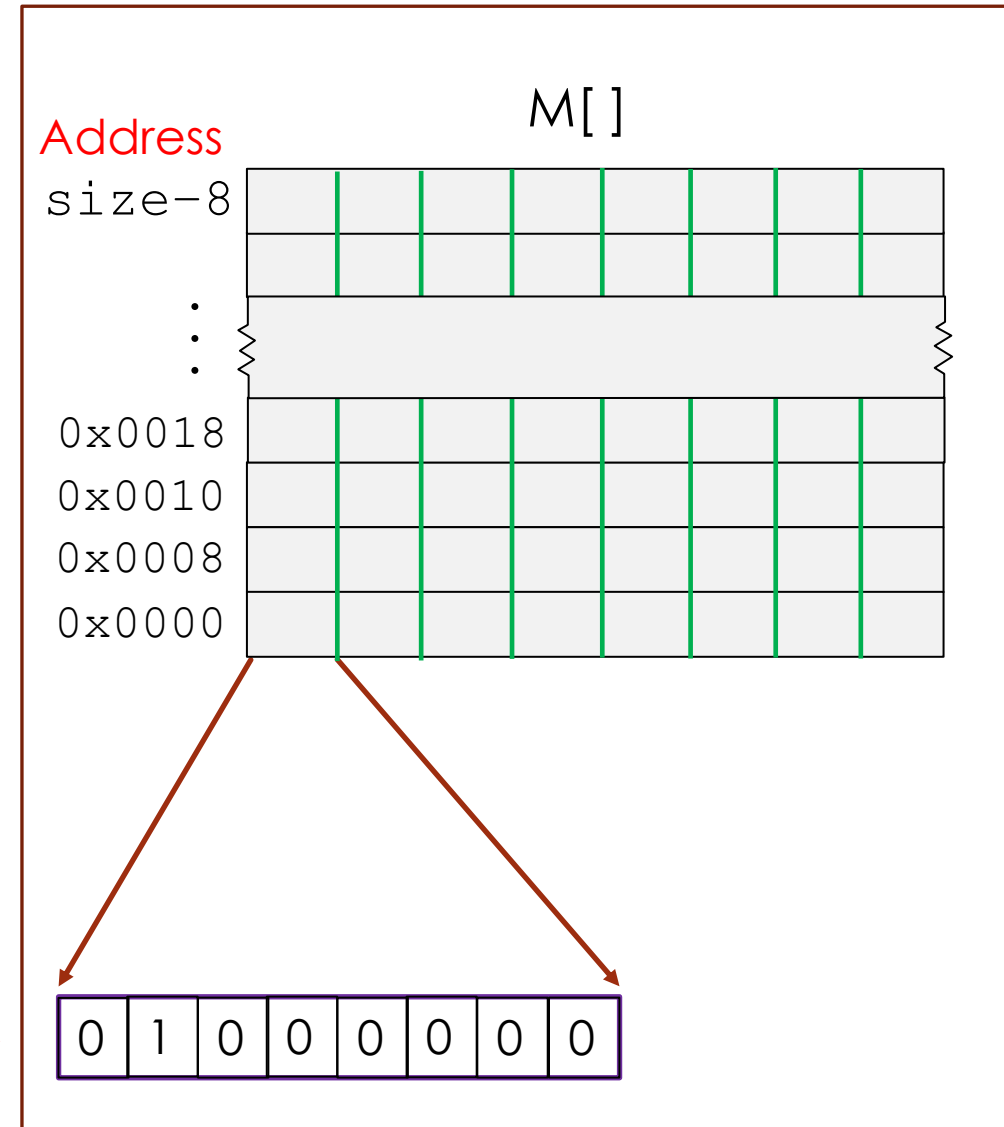
Compressed view of memory



- Question:**
1. If memory size is 2^6 bytes, then the memory addresses range from 0 to _____ ?
 2. In the diagram on the left, there is a byte with address **size** - 1. Where is this byte in the compressed diagram on the right?
 3. In the diagram on the right, should not the address **0x0010** actually be **0x0016**?

How to express memory content

- Typically, in a diagram depicting memory, we express the content of a **byte of memory** as a series of memory “**cells**” in which one of two possible values is stored
- These two possible values are represented using ‘0’ and ‘1’



Lab 0 - How to express memory content

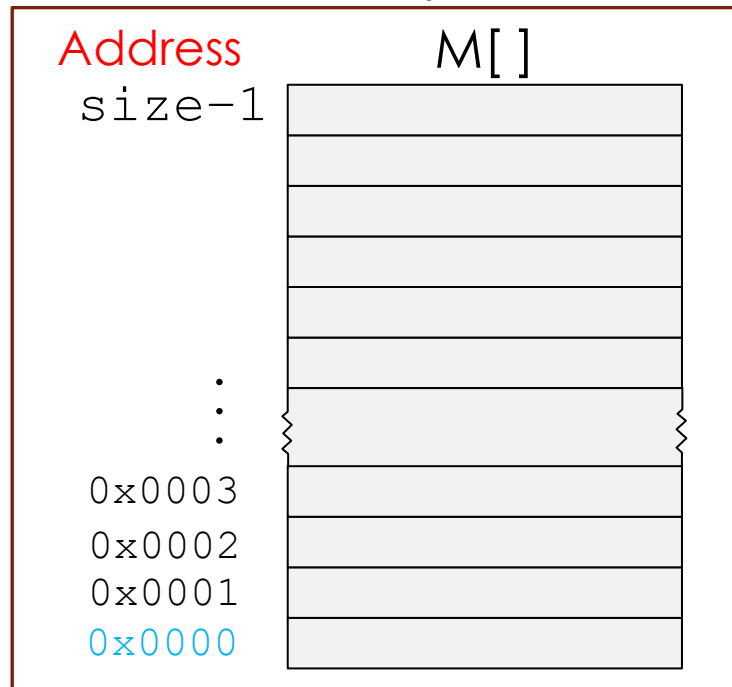
- Lab 0 – **Review** of the Binary numeral system,
the Hexadecimal numeral system
and the Binary $\Leftrightarrow$ Hexadecimal Conversion
- Practice how to express memory content
 - Using **binary** and **hexadecimal** numeral system
- Posted on our course web site
- Nothing to hand in
- Worth 0 marks
- Invitation to come to my office hours if you have questions

Endian

Order in which bytes stored in memory

- Determined by a microprocessor: how a microprocessor orders the bytes when it reads them from and writes them to memory.
- It is straightforward to store 1 byte of data in memory
 - All the microprocessor needs is the data (i.e., bit pattern) and a memory address
 - For example, let's store byte 01110011_2 at address $0x0000$

Least significant bit (**LSb**)?
Most significant bit (**MSb**)?



Endianness is not determined by software developers ☺. It is a characteristic of the microprocessor (ISA).

Endian

Order in which bytes stored in memory

Question: But how do we store several bytes in memory?

► For example, let's store these 4 bytes starting at address $0x0000$

01000010 01101001 01110100 01110011₂

Least significant
byte (**LSB**)?
Most significant
byte (**MSB**)?

Answer:

Way 1: Little endian

Address	M[]
size-1	
⋮	
0x0003	01000010
0x0002	01101001
0x0001	01110100
0x0000	01110011

Way 2: Big endian

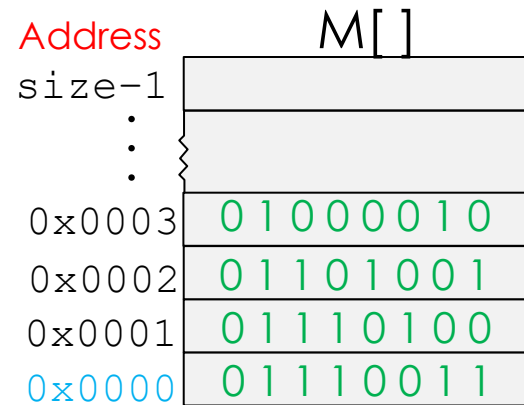
Address	M[]
size-1	
⋮	
0x0003	01110011
0x0002	01110100
0x0001	01101001
0x0000	01000010

Trick:

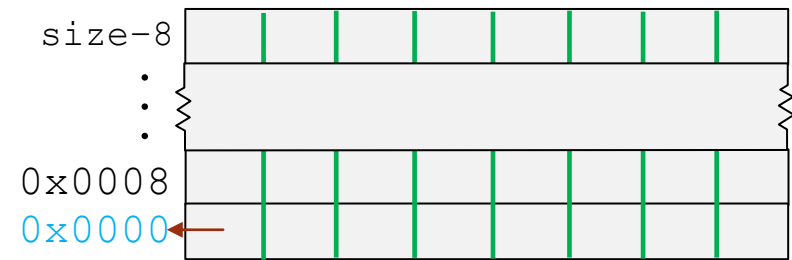
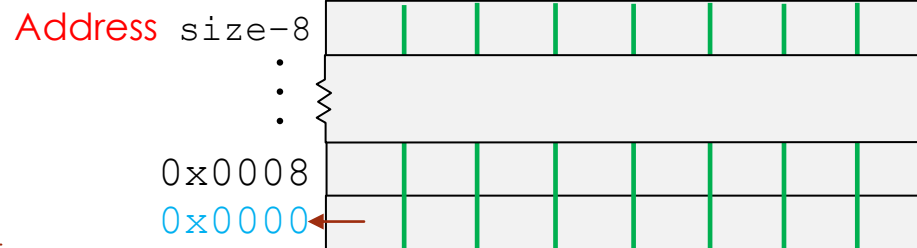
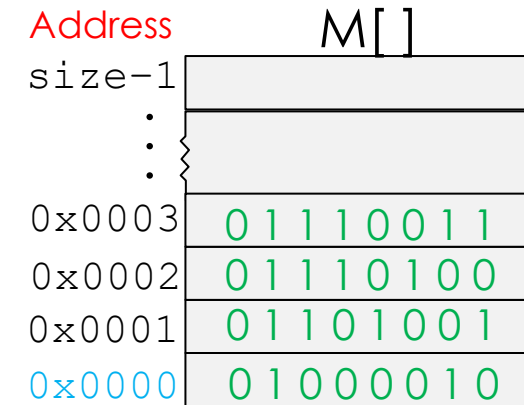
Endian

Order in which bytes stored in memory

Way 1: Little endian



Way 2: Big endian



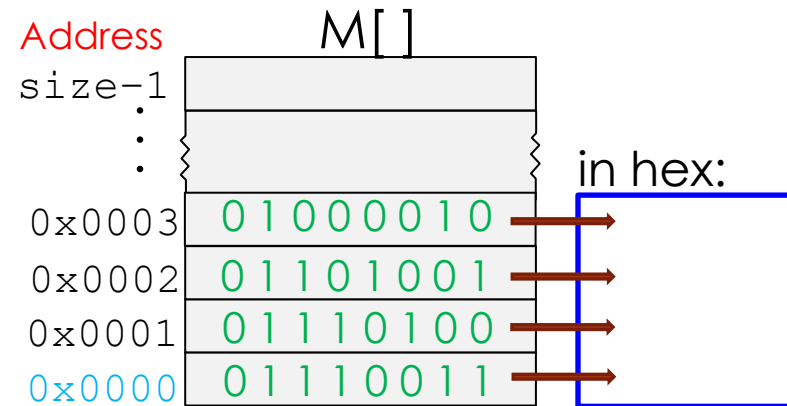
Compressed
view of memory

Endian

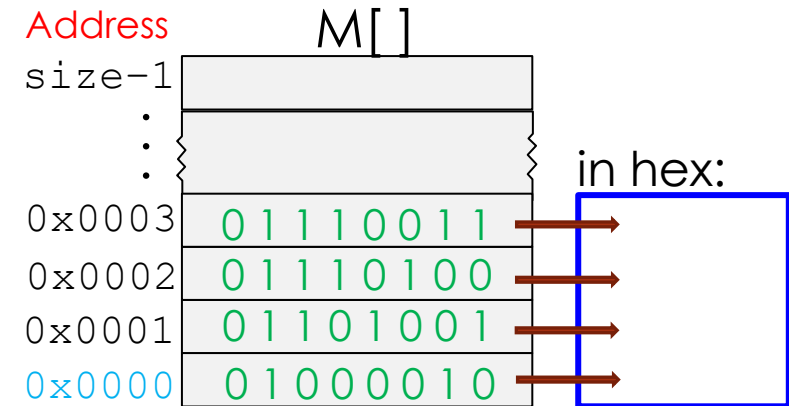
Order in which **bytes** stored in memory

- Convert the 4 bytes stored in memory into hexadecimal numbers and enter these hex numbers in each of the **blue** boxes.

Way 1: Little endian



Way 2: Big endian



Homework 1: Once you have done Lab 0, convert each byte stored in memory into its **hex** equivalent.

Review

No matter what data the bits in memory represent, these bits can be manipulated using **bit-level (bitwise) operations**:

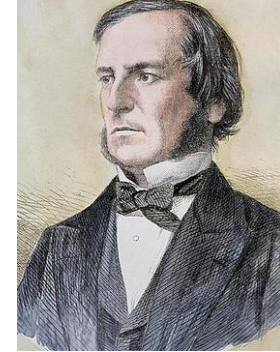
- Boolean algebra
- Shifting

Bit Manipulation: Boolean algebra

➤ Developed by **George Boole** in 19th Century

➤ Algebraic representation of logic

➤ Encode **true** as **1** and **false** as **0**



■ **AND** -> $A \& B = 1$ when both $A = 1$ and $B = 1$ ■ **OR** -> $A | B = 1$ when either $A = 1$ or $B = 1$

$\&$	0	1
0	0	0
1	0	1

$ $	0	1
0	0	1
1	1	1

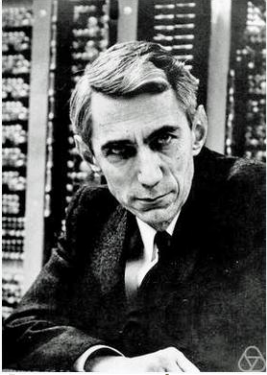
■ **NOT** -> $\sim A = 1$ when $A = 0$

$\sim$	
0	1
1	0

■ **XOR** (Exclusive-Or) -> $A \wedge B = 1$ when either $A = 1$ or $B = 1$, but not both

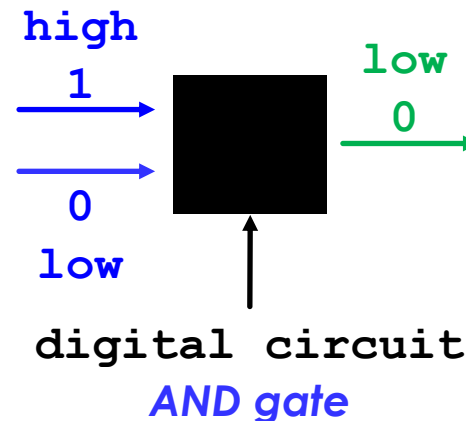
$\wedge$	0	1
0	0	1
1	1	0

Interesting fact about Boolean algebra and digital logic



- **Claude Shannon** – 1940 master's thesis (posted on our course web site)
- Made connection between **Boolean algebra** and **digital logic**
 - Boolean algebra could be applied to the design and the analysis of digital systems (digital circuits)

➤ Example:



Review Let's try some Boolean algebra!

- Operations applied **bitwise** -> to each pair of bits located at same position in two operands

Homework 2 Spot the error(s):

$$\begin{array}{r} 01101001_2 \\ \& 01010101_2 \\ \hline 01000001_2 \end{array}$$

$$\begin{array}{r} 01101001_2 \\ | 01010101_2 \\ \hline 01111101_2 \end{array}$$

$$\begin{array}{r} 01101001_2 \\ ^\wedge 01010101_2 \\ \hline 00111110_2 \end{array}$$

$$\begin{array}{r} \sim 01010101_2 \\ \hline 10101010_2 \end{array}$$

This slide can be used as **reference** throughout the semester!

Using Boolean algebra and *bit mask*

► **Bit mask:** Using a bit mask (binary mask) as one of the operands

1. **AND:** Extracts particular bit(s) from one of the operands (non-mask operand) so we can test whether the bit(s) are set or not

► Example:

10110011 ₂	<- some value x
& 00000001 ₂	<- bit mask
00000001 ₂	

i.e., set to 1

The result tells us that the least significant bit (LSb) of **x** is set

2. **XOR:** Toggle specific bits

► Example:

10110011 ₂	<- some value x
^ 00011100 ₂	<- bit mask
10101111 ₂	

We get a toggled version of the 3 original bits (of **x**) that correspond to the 3 set bits of the bit mask

► **Using two operands**

1. **OR:** Merge all set bits of operands

► Example:

10110011 ₂	<- some value x
 00011100 ₂	<- some value y
10111111 ₂	

The result contains all the set bits of **x** and **y**

Bit Manipulation: Shift operations

➤ Left Shift: $x \ll y$

➤ Shift bit vector x left
 y positions

a.k.a. bit pattern

➤ Effect:

1. Throw away y most significant bits (MSb) of x (on the left of x)
2. Fill x with y 0's on the right of x

- LSb: least significant bit is the rightmost bit of a bit vector
- MSb: most significant bit is the leftmost bit of a bit vector

➤ Right Shift: $x \gg y$

➤ Shift bit vector x right
 y positions

➤ Effect:

1. Throw away y least significant bits (LSb) of x (on the right of x)
2. Logical shift: Fill x with y 0's on the left of x

Arithmetic shift: Fill x with y copies of x 's **sign bit** on left

- **Sign bit**: most significant bit (MSb) of x (before shifting occurred)

Homework 3

Let's try some **Shift operations!**

1. Left Shift: $10111001_2 \ll 4$

2. Left Shift: $10111001_2 \ll 1$

3. Right Shift: $00111001_2 \gg 3$
logical

4. Right Shift: $10111001_2 \gg 5$
arithmetic

5. Right Shift: $10111001_2 \gg 2$
logical
arithmetic

Summary

- **von Neumann** architecture - architecture of most computers
 - Its components: CPU, memory, input and output, bus
 - One of its characteristics: Data and instructions (code/programs) both stored in memory
- A look at **memory**: defined **byte-addressable** memory, compressed view of memory
- A look at **memory content**
 - Why we abstract the content of a memory cell using the two values of the binary numeral system -> '0' and '1'
 - Algorithm for converting binary to hexadecimal (hex)
 1. Partition bit vector into groups of 4 bits, starting from right, i.e., least significant **byte** (LSB)
 - If most significant **byte** (MSB) does not have 8 bits, pad it: add 0's to its left
 2. Translate each group of 4 bits into its corresponding hex value
 - **little endian** versus **big endian**: Order in which bytes are stored in memory
- **Bit manipulation** – regardless of what a bit pattern (vector) represents/means
 - **Boolean algebra**: **bitwise operations** => **AND** (&), **OR** (|), **XOR** (^), **NOT** (~)
 - **Shift operations**: left shift $x \ll y$, logical and arithmetic right shift $x \gg y$
 - **Right logical shift**: Fill x with y 0's on the left of x
 - **Right arithmetic shift**: Fill x with y copies of x 's sign bit on the left of x
 - **Sign bit** is most significant **bit** (MSb) before shifting occurred

reviewed in Lab 0

Determined by a microprocessor. Describes how a microprocessor orders the bytes when it reads them from and writes them to memory.

NOTE:

C logical operators and C bitwise (bit-level) operators behave differently!
Watch out for && versus &, || versus |, ...

Next Lecture

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

Demo

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...

